

Relay-of-Experts: 一种面向消费级硬件的大模型串行专家接力架构

宇兴项目组

2026年3月18日

Abstract

我们关注一个很现实的部署问题：在固定小内存条件下，如何调用远超驻留上限的专家参数。为此，我们提出Relay-of-Experts (RoE) 架构。与需要并行驻留候选专家的Mixture of Experts (MoE) 不同，RoE 按时间顺序加载专家：任意时刻只保留一个当前专家（可选预取下一专家），跨专家上下文通过“黑板”中间状态传递。对应地，专家内存缩放规律由

$$M_{\text{MoE}}^{\text{expert}} = \sum_i P_i \quad (1)$$

转为

$$M_{\text{RoE}}^{\text{expert}} = \max_i(P_i) + M_{\text{board}}, \quad (2)$$

因此总运行内存可写为

$$M_{\text{RoE}}^{\text{total}} = M_{\text{backbone}} + M_{\text{RoE}}^{\text{expert}}. \quad (3)$$

我们不把“误差累积”当作边角问题，而是把它放在系统设计中心，并给出三层防护：置信度标注、共识校验与回滚恢复。结合当前95.6M 参数基座模型（YuxingLLM）与截至2026年3月18日的训练日志，文中给出RoE 的形式化定义、误差传播模型、实现路线与评估协议。我们在日志中观察到初步趋势：在多次断点恢复后，loss 从9.02 (step 10) 变化到4.43 (step 2120)。该观察仅用于说明训练流程可运行，不构成对最终效果的结论性证明。

1 引言

大模型能力通常随参数规模增长，但在消费级设备上，先撞上的往往不是算力墙，而是内存墙。即使采用MoE，部署时也常需要在同一时刻保留多个候选专家，峰值内存因此居高不下。我们在本地i5-10400 + 16GB DDR4 和云端CPU 8 核32GB 上训练95.6M 参数基座模型时，已经跑通了流式数据、词表统一和稳定训练流程；接下来更直接的问题是：在小内存不变的前提下，怎么把“可调用知识容量”做大？

这个约束并不是小场景问题。以FP16 权重粗估，500B 参数模型仅权重就约需1 TB；若单卡可用显存约80 GB，仅权重驻留就需要十余张加速卡，还没算KV cache 与激活开销。换句话说，在很多实际部署环境里，真正卡住系统的是“专家能否同时驻留”，而不是单步浮点算力。

我们要回答的问题是：当总专家容量可达500B，而设备同一时刻只能激活10B 甚至更小参数时，推理流程该如何组织？RoE 的做法是**串行专家接力**：把并行激活改成时间顺序激活，用黑板中间表示衔接多专家推理。直观地说，RoE 用可控时延换取可扩展容量，这也是本文后续实验重点检验的取舍关系。

在并行MoE 中，即便每个token 只激活top- k 专家，系统仍需维护候选专家可访问，峰值内存更接近 $N \cdot P_E$ 而非 $k \cdot P_E$ 。RoE 试图把该约束改写为“由最大单专家规模主导”，将容量扩展路径从并行同驻留转到时间维接力。

1.1 本文贡献

- **架构层面**：提出RoE 串行专家接力范式，并将其与“并行路由”路径放在统一框架下比较。
- **系统层面**：把专家内存需求从“专家参数求和”改写为“最大单专家+ 黑板开销”，并给出可工程实现的内存表达式与调度接口。
- **方法层面**：形式化误差累积过程，构建设信度、共识、回滚三层联动防护机制。
- **实验层面**：整理222 条训练日志并给出可复现实验协议（记录口径、恢复策略），为后续对比实验提供统一记录框架。

2 相关工作

与RoE 直接相关的研究大致可归纳为五类：基础语言模型组件、分词与子词建模、可扩展训练与数据实践、模块化专家扩展，以及黑板式协作推理。我们重点讨论与“内存受限下扩容”关系最密切的脉络。

基础语言模型组件。自回归语言建模主干可追溯至Transformer[1]，并在GPT 路线中得到系统化验证[2, 3]。位置编码方面，RoPE 是当前长上下文建模的常见选择[4]；激活函数方面，SwiGLU/GLU 变体已被证明有助于表达能力与训练稳定性[5]；注意力开销方面，GQA 通过共享K/V 缓解KV cache 负担[6]。这些组件并不直接解决专家驻留问题，但决定了基座模型在受限资源下的训练与推理效率。

分词与子词建模。本项目采用BPE 子词分词[7]。SentencePiece 提供语言无关的分词实现（含BPE/Unigram），可作为词表规范化与特殊符号管理的工程对照[8]。对RoE 而言，分词策略不仅影响压缩效率，也影响黑板中间状态的可表达性。

可扩展训练与数据实践。资源受限时，数据管线常常比模型结构更早成为瓶颈。我们采用流式读取并边读边tokenize 的out-of-core 路线，这与规模定律[9]和Chinchilla 的compute-optimal 观点[10]互补：在固定算力预算下，模型规模与数据规模需要协同优化。

模块化专家与容量扩展。参数扩容的一条主线是条件计算与MoE。早期“分治”思想可追溯至经典专家分配工作[20]；在语言模型中，稀疏门控MoE 通过按token 选专家显著提升容量[11]。随后，Switch Transformer 进一步简化路由[12]，GShard 与后续系统展示了分布式条件计算和自动分片的可扩展路径[13, 21]；近期Mixtral 也在中等规模上验证了MoE 的效果[22]。这条路线已经证明“条件计算可以扩容”，但大多数方案默认并行专家可访问，因此峰值内存仍受候选专家规模牵制。

RoE 与并行路由的关键差异在于：我们不再把“并行可访问”视为前提，而是把跨专家计算改写为时间序列。这一思路与模块化/可组合子网络研究方向相近，例如Progressive Networks 通过冻结旧模块并新增模块实现知识累积[14]，PathNet 通过路径选择与子网络复用实现扩展[15]。

专家卸载与分页（offloading/paging）。在不改变MoE 并行计算范式的前提下，已有工作通过CPU/SSD 等分层存储实现专家卸载和按需加载[23, 24, 25, 26]。这类方法主要回答“并行路由下如何更聪明地管理专家驻留”；RoE 进一步追问“是否必须并行驻留”，因此属于更上游的计算图重排。

黑板式协作与分步推理。本文采用的黑板（Blackboard）机制可追溯至经典Blackboard Systems[16]，并与更早体系（如HEARSAY-II）在“共享工作区+ 多知识源协作”上保持一致[27, 28]。在现代LLM 系统中，显式中间状态和分步决策也常用于提升复杂任务表现：Chain-of-Thought 强化推理链条[29]，多智能体辩论提升答案稳健性[30, 31]，ReAct 将推理与行动交错组织[17]。与这些多在应用层的方法不同，RoE 将“分步协作”前移到架构层，并与调度、回滚和内存预算联合设计。

Pipeline 并行与模型切分。模型并行沿“按层切分”的pipeline 和“按张量切分”的路径演进，例如GPipe[32]、PipeDream[33] 和Megatron-LM[34]。这类方法主要解决吞吐和单体模型

扩展问题，并不直接构造“可交换专家池”。因此，Pipeline 更像“把一个大模型切开跑”，RoE 更像“让多个专家按时间接力跑”。

若后续专家化采用参数高效微调（PEFT），则可参考LoRA 与Adapter 等方法以降低微调成本[18, 19]。

综上，RoE 与既有工作的关键差异不在于“分页技巧”或“路由细节调参”，而在于计算图组织方式本身：我们将跨专家执行重构为时间串行流程，把系统设计焦点从“并行专家如何同驻留”转到“中间状态如何稳定、可验证地跨专家传递”。

3 相关范式与问题定义

3.1 与MoE、Pipeline 的区别

- **MoE（并行路由）**：对每个token 选择top- k 专家，重点是条件计算；核心挑战是并行激活与通信。
- **Pipeline Parallelism（模型切分）**：把单个大模型切到多设备，重点是吞吐和流水调度；并不增加可交换专家池。
- **RoE（串行专家接力）**：专家在时间维接力执行，黑板承载中间状态；重点是单机内存受限下的容量扩展。

3.2 问题定义

设专家集合为 $\mathcal{E} = \{E_1, \dots, E_N\}$ ，第 i 个专家参数为 P_i 。在传统常驻式方案下，峰值专家内存近似：

$$M_{\text{peak}}^{\text{resident}} \approx M_{\text{backbone}} + \sum_i P_i \cdot b, \quad (4)$$

其中 b 为每参数字节数。RoE 的目标是在固定内存预算 M_{budget} 下最大化可用专家总容量，并保持可接受时延。其关键约束写为：

$$M_{\text{peak}}^{\text{RoE}} \approx M_{\text{backbone}} + a \cdot \max_i(P_i) \cdot b + M_{\text{board}} \leq M_{\text{budget}}, \quad (5)$$

其中 $a \in \{1, 2\}$ 表示是否启用单步预取。

4 Relay-of-Experts 架构

4.1 核心机制：黑板驱动的时间串行专家

RoE 推理可以理解“路由器+ 专家+ 黑板”的闭环协作：

- **Router**：根据任务状态选择下一位专家；
- **Expert**：对当前上下文执行子任务并写回结果；
- **Blackboard**：保存结构化中间状态（痕迹、证据、局部结论、置信度）。

为保证可复现性，我们将黑板条目定义为

$$b_t = (\tau_t, c_t, \phi_t, \pi_t, t, id_t), \quad (6)$$

其中 τ_t 表示条目类型（如FACT/RESULT/ERROR_FLAG）， c_t 为内容载荷， $\phi_t \in [0, 1]$ 为置信度， π_t 为来源专家标识， t 为接力步号， id_t 为唯一条目索引。黑板状态记为

$$\mathcal{B}_t = \{b_1, b_2, \dots, b_{|\mathcal{B}_t|}\}. \quad (7)$$

基于该状态，系统提供三类原语：READ（按类型/置信度过滤读取）、WRITE（追加新条目）与ANNOTATE（对既有条目标注VERIFIED 或DISPUTED 状态）。

给定输入 x ，Router（或轻量分解器）生成专家序列

$$\sigma = \mathcal{D}(x) = [\sigma(1), \sigma(2), \dots, \sigma(n)], \quad (8)$$

并在每步执行

$$o_t = E_{\sigma(t)}(\text{READ}(\mathcal{B}_{t-1}, q_t)), \quad \mathcal{B}_t = \text{WRITE}(\mathcal{B}_{t-1}, o_t). \quad (9)$$

对开放式任务，可进一步采用动态调度 $\sigma(t+1) = \mathcal{D}_{\text{dyn}}(\mathcal{B}_t)$ 。

直观来看，每一跳都执行同一循环：加载专家→ 读取黑板→ 处理输入→ 写入黑板→ 卸载专家，再进入下一位专家。图 1 给出时间线示意。

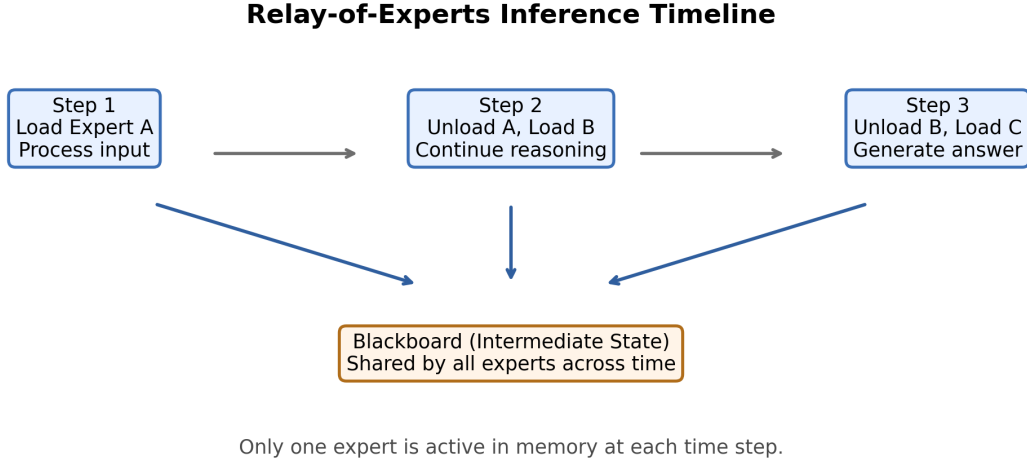


Figure 1: Relay-of-Experts 的串行接力推理流程。任意时刻仅激活一个专家，跨专家状态通过黑板传递。

4.2 内存与容量关系

在RoE 中，峰值专家内存近似：

$$M_{\text{peak}}^{\text{RoE}} \approx M_{\text{backbone}} + a \cdot P_E \cdot b + M_{\text{board}}, \quad (10)$$

其中 $a \in \{1, 2\}$ ，分别对应单专家激活与“当前专家+ 预取专家”。因此，当专家数 N 增加时，RoE 的峰值内存近似保持常量，增长主要转移到磁盘占用和调度成本。图 2 展示了这一趋势。

4.3 一个最小例子：计算1+1

该任务可分解为数字语义识别、运算符语义识别和求值三个子步骤：

1. 数字专家A 写入“1 表示单个单位”；

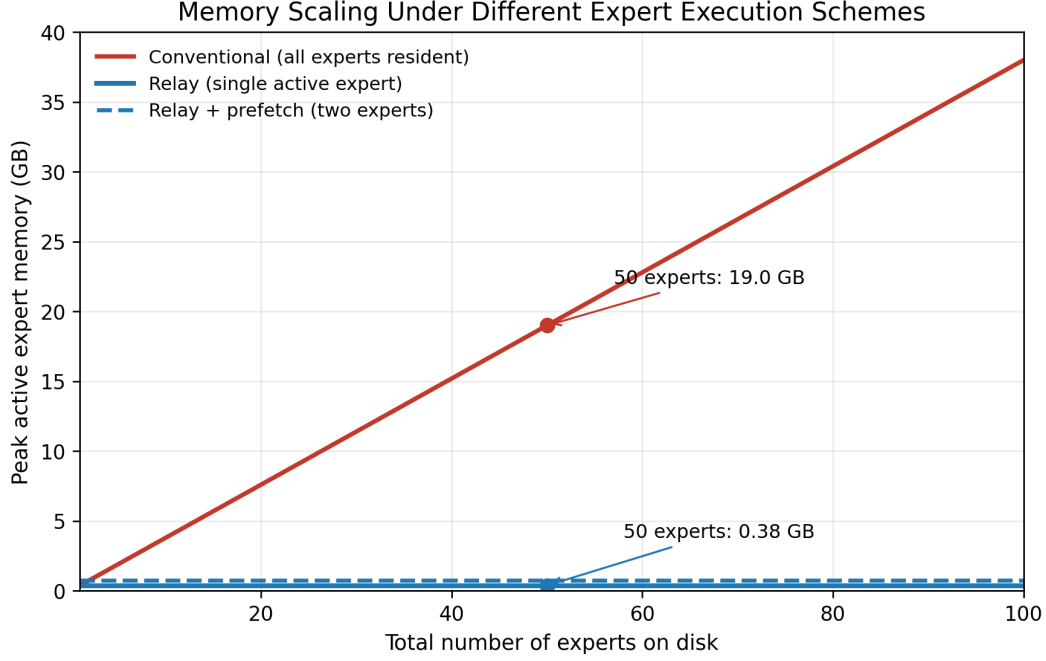


Figure 2: 专家数量增长时的峰值专家内存。常驻式方案随 N 线性增长；RoE 近似常数。

2. 运算符专家B 写入“+ 表示求和”；
3. 计算专家C 读取黑板并输出“ $1 + 1 = 2$ ”。

这个例子虽然简单，但它清楚说明了一点：即使任意时刻只激活一个专家，只要中间状态可读、可写、可追踪，多步协作依然能形成稳定的推理链。

5 误差累积分析与三层缓解机制

5.1 误差传播建模

可将串行接力看作“信息接力赛”：早期步骤的偏差会被后续步骤继续读取和放大。设接力链为 $E_1, \dots, E_n$ ，第 t 步理想输出为 y_t^* ，实际输出为 y_t ，定义误差 $\epsilon_t = y_t - y_t^*$ ，则有

$$\epsilon_t = \delta_t + f_t(\epsilon_1, \dots, \epsilon_{t-1}), \quad (11)$$

其中 δ_t 是专家自身误差， f_t 刻画上游误差对当前步骤的影响。若近似满足 $|\epsilon_t| \leq \delta + \gamma|\epsilon_{t-1}|$ ，当 $\gamma > 1$ 时误差会随链长增长。这就是RoE 在系统层必须正面处理、不能回避的问题。

5.2 Tier-1: 置信度标注

每个专家除内容外同步写入置信度 $\phi_t \in [0, 1]$ 。下游读取时按置信度加权，并在 $\phi_t < \phi_{\min}$ 时触发复核路径。该层几乎不增加额外加载次数，是成本最低、覆盖面最广的一层防护。

5.3 Tier-2: 多专家共识校验

对关键子任务引入 m 个候选专家独立求解，再用多数投票或置信度加权聚合。该层通常带来最明显的正确率增益，但也会引入约 m 倍子任务时延，因此更适合由Tier-1 的低置信信号触发、按需稀疏调用。

5.4 Tier-3: 检查点回滚恢复

每步专家执行后保存黑板检查点，最终答案再由轻量验证器判定是否接受。若拒绝，则定位首个错误步骤并回滚到对应检查点，改用替代专家或替代解码策略重试。该层将“单次链路失败”转化为“有限轮可恢复”，是长链任务的兜底机制。

6 当前工程基础与可行性

6.1 基座模型现状

当前基座模型为YuxingLLM（Decoder-only Transformer, GQA + RoPE + SwiGLU），参数量95.6M，词表规模26197。训练配置为序列长度512、全局batch size 32、最大学习步数50,000、warmup 2,000、峰值学习率 3×10^{-4} 。已完成的关键修复包括词表统一、以流式数据集替换全量tokenize、以及训练日志与故障定位增强。这些工作看似“底层工程”，但直接决定了后续RoE 专家化实验能否稳定推进。

6.2 已有训练观测

图 3 基于训练数据.txt 的222 条日志记录绘制（统计区间为2026年3月17日至2026年3月18日）。为避免断点续训导致的重复步数干扰，主曲线对相同步数保留最新记录，并以浅灰散点保留原始日志。

从日志可见，loss 在step 10 到step 2120 区间内出现由9.02 到4.43 的变化，ppl 也出现相应下降。图中三条灰色虚线对应checkpoint 恢复点（step 510、1210、2110）；恢复后曲线仍可继续推进。该现象可视为流程可运行的初步观察，不构成最终性能结论。与此同时，step 1450 附近出现“短暂低谷后回升”，后续仍需结合验证集指标和学习率策略做更细致的稳定性诊断。

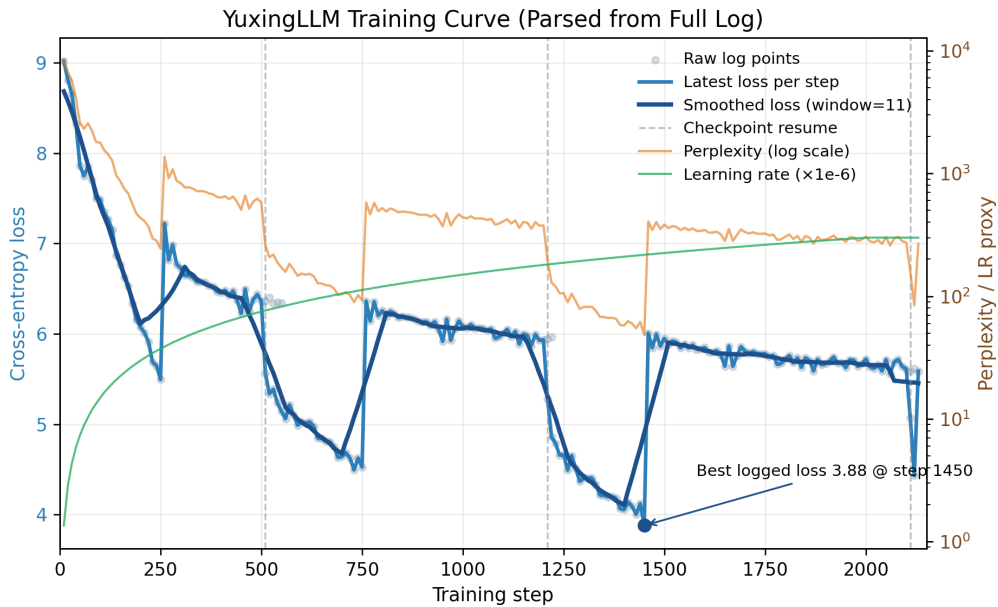


Figure 3: YuxingLLM 训练日志曲线（2026年3月17日至2026年3月18日）。蓝色为按step 去重后的loss，深蓝为平滑趋势，橙色为ppl（对数轴），绿色为学习率（缩放后），灰色虚线为断点恢复位置。

7 实验设计、初步观察与评估协议

我们将评估目标分成两组：一组关注任务能力相关指标，另一组关注内存与时延等系统指标。本文在该部分主要给出评估框架和日志观察，不对模型效果作定论。

7.1 对比基线

基线选择遵循一个原则：覆盖“常驻专家池”“并行稀疏路由”“模型并行切分”三种主流扩容路径，以便清楚区分RoE 的优势来源究竟来自路由策略，还是来自计算图重排。

Table 1: RoE 与常见方案的对比维度

方案	专家激活方式	峰值内存随专家数	主要代价
Dense/常驻专家池	同时常驻	线性增长 $O(N)$	内存占用高
MoE (top- k)	同步激活多个专家	依赖并行专家驻留	路由与通信复杂
Pipeline 并行	模型切分跨设备	由分片策略决定	部署与调度复杂
RoE (本文)	时间串行接力	近似常数	时延上升

7.2 核心指标

考虑到RoE 的本质是“以内存换时延的结构重排”，指标设计需要同时覆盖质量、系统效率与鲁棒性三条线。

- **质量指标**：验证集困惑度、下游任务准确率、长链推理成功率；
- **系统指标**：峰值内存、端到端时延、吞吐、专家切换频率；
- **鲁棒性指标**：黑板噪声注入后性能退化、专家缺失时降级能力。

7.3 复现实验设置

- **硬件环境**：本地i5-10400 + 16GB DDR4；云端CPU 8 核+ 32GB（含中断恢复场景）；
- **训练配置**：seq_len=512, global_batch=32, max_steps=50,000, warmup=2,000, 峰值学习率 3×10^{-4} ；
- **日志口径**：按固定步频记录step/loss/ppl/lr/tok/s/memory，并周期性保存checkpoint；
- **恢复协议**：从最近checkpoint 恢复并显式标注恢复点，保留恢复前后曲线用于稳定性分析。

7.4 消融实验建议

- **黑板表示**：自然语言摘要vs 向量槽位vs 键值状态；
- **调度策略**：固定顺序vs 规则路由vs 学习路由；
- **专家粒度**：任务型专家（数字/运算/事实）vs 领域型专家（代码/数学/写作）；
- **预取机制**：无预取（ $a = 1$ ）vs 单步预取（ $a = 2$ ）。

7.5 训练日志初步观察（非结论性）

基于训练数据.txt，当前共统计222条训练记录。为确保可比性，我们同时报告原始日志口径与恢复段口径，结果见表2。

Table 2: YuxingLLM 训练日志观察摘要（2026年3月17日至3月18日）

指标	数值	说明
原始日志条数	222	全部step/loss/ppl记录
去重后有效step	213（10–2130）	相同步数保留最新记录
初始loss	9.0204（step 10）	训练启动阶段
观测最低loss	3.8784（step 1450）	对应ppl=48.3
最新恢复段末值	4.4307（step 2120）	最后一次恢复后的连续片段
平均吞吐	439.7 tok/s	区间417–464 tok/s
恢复点	step 510/1210/2110	三次断点恢复事件

从日志记录看，在多次中断恢复后曲线仍可继续推进：loss由9.0204变化到3.8784。该现象可作为“训练流程可继续运行”的初步信号，但尚不足以支持关于最终性能的结论。

同时我们也注意到一个值得跟踪的细节：step 1450附近出现“短暂低谷后回升”。在现有日志粒度下，该波动可能与恢复后优化状态重建、学习率相位或数据切片分布有关；具体原因仍需通过验证集同步评估与更细粒度状态记录进一步核查。

当前版本尚未开展跨模型、跨任务的系统对比实验，也未完成链长消融实验。后续将按同一硬件与同一评测口径补充这些结果。

8 工程落地路线

工程实现建议按以下模块推进：

- `relay_board.py`：定义黑板状态结构、序列化与压缩接口；
- `relay_router.py`：定义专家选择策略与停止条件；
- `relay_finetune.py`：在统一基座上微调不同专家；
- `relay_inference.py`：实现接力推理主循环与日志追踪。

建议先完成最小闭环（3个专家+算术任务），确认端到端流程、日志和回滚逻辑都稳定，再扩展到多领域专家池，并记录每步黑板轨迹用于误差分析。

9 局限性与风险

RoE的目标是缓解内存压力，但它也可能把一部分负担转移到时延与调度复杂度。现阶段主要风险如下：

- **时延问题**：串行接力天然增加首token延迟；
- **误差累积**：黑板中间状态若偏移，后续专家可能放大误差；
- **路由稳定性**：专家选择器可能出现循环或早停失败；
- **训练成本**：需要额外专家化微调与系统级调优。

从工程角度看，以上风险并非彼此独立。比如路由不稳定会放大误差累积，误差累积又会触发更多回滚，进一步抬高端到端时延。我们后续将优先做两件事：其一，建立“链长-错误率-时延”联合监控；其二，在低置信触发阈值上做系统级网格搜索，以控制质量与时延的折中点。

10 结论

RoE 给出了一条面向小内存硬件的可行路线：通过“专家串行+ 黑板中间态”，将可用参数总量的主要约束从内存迁移到存储。其关键价值在于把专家内存需求从“参数求和”改写为“最大单专家+ 黑板开销”，让消费级设备具备承载高容量专家池的可能。

本文目前形成了“理论定义-系统设计-阶段性实测-评估协议”的闭环：

- 在理论层面，形式化了串行专家链的误差传播并提出三层缓解机制；
- 在系统层面，给出了可实现的黑板原语与专家调度表达；
- 在观察层面，整理并报告了222 条训练日志，形成可追踪的记录口径；
- 在评估层面，给出了后续对比实验所需的统一记录口径与扩展步骤。

因此，RoE 并非MoE 的替代，而是容量扩展维度上的互补：**并行路由更偏吞吐优化，串行接力更偏容量优化**。当前稿件的定位是“方法定义+ 机制说明+ 评估框架”，不对最终效果作结论性陈述。下一步关键工作将聚焦在跨任务泛化、时延上界控制和黑板表示压缩三个方向。

学术诚信与AI辅助声明

本文在写作与语言润色阶段使用了大语言模型工具进行表达优化与格式检查。所有技术路线、实验记录、结果解释与结论判断均由作者团队独立核实并承担责任；文中引用文献亦由作者逐条核对，不将AI 工具列为论文作者。

References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention Is All You Need. In NeurIPS, 2017.
- [2] Alec Radford, Jeffrey Wu, Rewon Child, et al. Language Models are Unsupervised Multi-task Learners. OpenAI Technical Report, 2019.
- [3] Tom B. Brown, Benjamin Mann, Nick Ryder, et al. Language Models are Few-Shot Learners. In NeurIPS, 2020.
- [4] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864, 2021.
- [5] Noam Shazeer. GLU Variants Improve Transformer. arXiv:2002.05202, 2020.
- [6] Joshua Ainslie, Tao Lei, Michiel de Jong, et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. arXiv:2305.13245, 2023.
- [7] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural Machine Translation of Rare Words with Subword Units. In ACL, 2016.

- [8] Taku Kudo and John Richardson. SentencePiece: A simple and language independent sub-word tokenizer and detokenizer for Neural Text Processing. In EMNLP: System Demonstrations, 2018.
- [9] Jared Kaplan, Sam McCandlish, Tom Henighan, et al. Scaling Laws for Neural Language Models. arXiv:2001.08361, 2020.
- [10] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. Training Compute-Optimal Large Language Models. arXiv:2203.15556, 2022.
- [11] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, et al. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In ICLR, 2017.
- [12] William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. arXiv:2101.03961, 2021.
- [13] Dmitry Lepikhin, HyukJoong Lee, Yuanzhong Xu, et al. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. In ICLR, 2021.
- [14] Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, et al. Progressive Neural Networks. arXiv:1606.04671, 2016.
- [15] Chrisantha Fernando, Dylan Banarse, Charles Blundell, et al. PathNet: Evolution Channels Gradient Descent in Super Neural Networks. arXiv:1701.08734, 2017.
- [16] Robert Englemore and Tony Morgan (eds.). Blackboard Systems. Addison-Wesley, 1988.
- [17] Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629, 2023.
- [18] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685, 2022.
- [19] Neil Houlsby, Andrei Giurugu, Stanislaw Jastrzebski, et al. Parameter-Efficient Transfer Learning for NLP. In ICML, 2019.
- [20] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive Mixtures of Local Experts. Neural Computation, 3(1):79–87, 1991.
- [21] Nan Du, Yanping Huang, Andrew M. Dai, et al. GLaM: Efficient Scaling of Language Models with Mixture-of-Experts. In ICML, 2022.
- [22] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, et al. Mixtral of Experts. arXiv:2401.04088, 2024.
- [23] Samyam Rajbhandari, Yuxiong He, and Fang Liu. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. arXiv:2201.05596, 2022.
- [24] Zhihong Shen, Zeqi Lin, Hao Cheng, et al. SE-MoE: Memory-Efficient Mixture-of-Experts with Shared Experts. arXiv preprint, 2023.
- [25] Xiaonan Nie, Rui Wang, and Zhiqiang Shen, et al. FlexMoE: Scaling Large-scale Sparse Models on Heterogeneous Hardware. arXiv preprint, 2023.
- [26] Wonjae Hwang, Seonghyeon Kim, and Sung Ju Hwang, et al. Pre-gated Mixture-of-Experts: Predictive Expert Prefetching for Efficient Inference. arXiv preprint, 2024.

- [27] H. Penny Nii. Blackboard Systems: The Blackboard Model of Problem Solving and the Evolution of Blackboard Architectures. *AI Magazine*, 7(2):38–53, 1986.
- [28] Lee D. Erman, Frederick Hayes-Roth, Victor R. Lesser, and D. Raj Reddy. The Hearsay-II Speech-Understanding System: Integrating Knowledge to Resolve Uncertainty. *ACM Computing Surveys*, 12(2):213–253, 1980.
- [29] Jason Wei, Xuezhi Wang, Dale Schuurmans, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *NeurIPS*, 2022.
- [30] Yilun Du, Sherry Yang, Bo Dai, et al. Improving Factuality and Reasoning via Multi-Agent Debate. *arXiv:2305.14325*, 2023.
- [31] Percy Liang, Tianjun Zhang, and others. Collaborative Multi-Agent Reasoning with Large Language Models. *arXiv preprint*, 2023.
- [32] Yanping Huang, Youlong Cheng, Ankit Bapna, et al. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. In *NeurIPS*, 2019.
- [33] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, et al. PipeDream: Generalized Pipeline Parallelism for DNN Training. In *SOSP*, 2019.
- [34] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, et al. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv:1909.08053*, 2020.